

COMP226 Assignment 2: Strategy Development with BackTrader

Continuous Assessment

Continuous Assessment Number	2 (of 2)
Weighting	10%
Assignment Circulated	Week 9
Deadline	8th May 2026 17:00 (Week 12).
Submission Mode	Submit up to two files to the CodeGrade assignment on Canvas: <code>solution.py</code> (required to get marks) and <code>pair_trading_solution.py</code> (optional).
Learning Outcomes Assessed	This assignment addresses the following learning outcomes: • Understand the spectrum of computer-based trading applications and techniques, from profit-seeking trading strategies to execution algorithms. • Be able to design trading strategies and critically evaluate their historical performance and robustness. • Understand the common pitfalls in developing trading strategies with historical data.
Summary of Assessment	• The goal is to implement and optimize three trading strategies (i.e., TMA, BB overbought/oversold, pair trading) within the BackTrader framework. • Marks are available for the correct implementation of 8 functions in <code>template.py</code> for TMA and BB overbought/oversold (70%). • Further marks (that require a correct implementation in <code>template.py</code>) are available for implementing pair trading (30%). • CodeGrade pre-deadline will be available for you to check the correctness of your work.
Submission necessary in order to satisfy module requirements	No
Expected time taken	Roughly 8–12 hours
Module Coordinator (Contact)	Andrew Roxburgh

Late Submission

Standard UoL policy applies; every student may submit up to 7 calendar days after the deadline, without penalty. Students who have a Student Support Information Sheet (SSIS) that includes the “coursework deadlines extensions allowed” adjustment can make use of a further 7-day extension period (total of 14 calendar days after the deadline). **Any attempt to submit more than 7 days (or 14 days with an adjustment) after the deadline will be classed as non-submission, and will receive a mark of zero.**

Warning

Submissions are automatically put through a plagiarism and collusion detection system. Students found to have plagiarised or colluded will likely receive a mark of zero. Do not discuss or show your work to others, and do not search for solutions to the assignment online. In previous years, students have had their studies terminated and left without a degree because of plagiarism or collusion.

Python

In this assignment, we use Python to run our code, e.g.,

```
python main.py
```

General instructions and assumptions

- You should use `yfinance` version 1.1.0 or later.
- You may modify `main.py` for your own debugging and testing, but only your submitted `solution.py` and, optionally, `pair_trading_solution.py` will be marked.
- Throughout this assignment, assume that strategies do **not** accumulate inventory. That is, once you are in a position, you should only consider the corresponding exit logic; you should not keep entering additional units while the same signal continues.
- For Task 7, Task 8, and Extra 4, it is acceptable to obtain the required values manually by running the relevant code and then return the final result as a hard-coded dictionary. You are not required to automate the final selection process unless explicitly stated.

Distributed code

As a first step, please download `comp226_a2.zip` via the assignment page on Canvas. Then unzip `comp226_a2.zip`, which will yield the following contents in the directory `comp226_a2`:

```
comp226_a2
- get_periods.py
- main.py
- pair_trading_template.py
- template.py
```

Brief summary

You are provided with four `.py` files, two complete:

- `main.py` is the file you will run with Python. By running `main.py`, you will get the outputs of the functions you write. It can also be used for debugging if any bug appears. You can edit `main.py` for debugging purposes.

- `get_periods.py` is the file you will use to get your specific trading data period. This period is different for each student.

and two incomplete files:

- `template.py` is the file that you will edit – the distributed version contains empty functions. It contains 8 empty functions.
- `pair_trading_template.py` is the file that you will edit for the “extra” problem.

You need to change the file name from `template.py` to `solution.py` first. Then it will work with `main.py`. `solution.py` is also the file name you need to submit. Similarly, you need to change `pair_trading_template.py` to `pair_trading_solution.py` before submission.

To get 70%, you will need to correctly complete all 8 of these functions in `template.py`; if your answer is only partially correct, you will get a mark less than 70%. If you have correctly completed all 8 of these functions, you can then – and only then – get marks for the “extra” function, which accounts for 30% of the marks.

You should submit a Python file that contains your implementation of some or all of these 8 functions in `template.py` and, optionally, a Python file for the “extra” problem (`pair_trading_template.py`). Your submission will be marked automatically by CodeGrade.

- The tests for the 8 functions in `template.py`, which give up to 70% of the marks, will run at the time of submission and are fully visible pre-deadline on CodeGrade (detailed guidance on using CodeGrade to improve your mark can be found at the end of this document);
- The tests for the “extra” problem will only run post-deadline, and only if you got full marks for the functions in `template.py`.

You can (and, if required, should) submit multiple times to repeatedly use the CodeGrade pre-deadline tests; for a correct solution, CodeGrade will show that you pass all tests and have thus achieved the first 70% of marks.

In this assignment, every student has their own trading data periods for Apple (AAPL) based on their MWS username (only the part before the @, e.g., for Andrew Roxburgh, this username is `andyrox`, rather than the full email form `andyrox@liverpool.ac.uk`). By having different time periods, there is not one single correct answer for some tasks of this assignment. To get your periods, you need to run `get_periods.py` with your username.

```
python get_periods.py andyrox
```

Running `get_periods.py` will give you a start date and an end date.

Part 1: Strategy implementation with BackTrader (70%)

In `template.py`, we aim to implement and optimize the Triple Moving Average (TMA) strategy and the Bollinger Band Overbought/Oversold strategy using BackTrader. The implementation has been split into 8 separate tasks in `template.py`. We will use Apple (“AAPL”) as our testing stock. The description of each task is shown below.

Task 1: Download and process the trading data from Apple (5%)

In this task, you are asked to download the trading data of `ticker_symbol` using `yfinance`. You also need to specify the start and end date when downloading the data. Once downloaded, you are asked to create a BackTrader data container `bt.feeds.PandasData` and feed in the data downloaded from `yfinance`. The container should include the open, high, low, close, and volume

of the data. The close price should correspond to the adjusted close price from the trading data, but should fall back to the Close price if the Adjusted Close price is not available. So you need to link the columns in the container and the trading data correctly. Finally, you should return the container.

Task 2: Create a Cerebro (5%)

In this task, you are asked to create a cerebro using BackTrader. After initializing the cerebro, you need to add data, set the initial cash, commission fee, and slippage percentage properly. The data, cash, commission, and slippage percentage have been given as arguments to the function. Finally, you should return the cerebro.

Task 3: Implement the TMA strategy (15%)

In this task, you are asked to implement the TMA Strategy using BackTrader. In the `task_3` function, you ONLY need to complete the `__init__` and `next` functions. In the `__init__` function, you are asked to initialize BackTrader indicators for computing simple moving averages with different lookback windows. You are encouraged to use existing indicators in the BackTrader package. The lookback windows with different lengths are the default parameters `short_period`, `medium_period`, and `long_period` given in the class. The computation of simple moving averages should be based on the close price from the underlying cerebro data (i.e., the adjusted close price from the original trading data), as we demonstrated in class. In the `next` function, you need to implement the logic of the TMA strategy, defining the entry conditions for buying and selling. The exit condition will be a trailing stop loss with parameter `trailpercent=0.02` for both long and short positions. Refer to this page for the trailing stop loss:

<https://www.backtrader.com/blog/posts/2017-03-22-stoptrail/stoptrail/>

When you compare the relationship among the three moving averages, use greater than (`>`) or less than (`<`). For buying and selling, you don't need to specify the quantity of each trade right now. We will do it globally later.

In addition, in this task, the logging function, `notify_order` function, and `notify_trade` function are provided. These functions can be helpful for debugging. But if you don't want extra trade information to be printed, you can comment out these functions, which will not affect the test results.

Task 4: Implement the BB overbought/oversold strategy (15%)

In this task, you are asked to implement the Bollinger Band Overbought/Oversold Strategy using BackTrader. In the `task_4` function, you ONLY need to complete the `__init__` and `next` functions. In the `__init__` function, you need to initialize the Bollinger Band indicator using the default parameters `period` (i.e., the lookback window) and `devfactor` (i.e., the number of standard deviations for the width of the band). You are encouraged to use existing indicators in the BackTrader package. The computation of the Bollinger Band should be based on the close price from the underlying cerebro data (i.e., the adjusted close price from the original trading data), as we demonstrate in class. In the `next` function, you need to implement the logic of the Bollinger Band Overbought/Oversold strategy, defining the entry conditions for buying and selling. The exit condition is that you take the opposite position when the current price crossed the band. That is, if you are in a long position, then go flat when the price crosses the upper band. If you are in a short position, then go flat when the price crosses the lower band. When you compare the relationship between the moving average and the bands, use greater than (`>`) or less than (`<`). For buying and selling, you don't need to specify the quantity of each trade right now. We will do it globally later.

In addition, in this task the logging function, `notify_order` function, and `notify_trade` function are provided. These functions can be helpful for debugging. But if you don't want extra

trade information to be printed, you can comment out these functions, which will not affect the test results.

Task 5: Add a strategy and set the sizer (5%)

In this task, you are asked to add a strategy to a cerebro and adjust the sizer per trade. The cerebro, strategy, and size have been given as arguments to the function. Finally, you should return the cerebro.

Task 6: Run the cerebro (5%)

In this task, you are asked to run the cerebro and return the cash after running the cerebro (i.e., the cash you have at the end of the backtesting). The cerebro has been given as an argument to the function.

Task 7: Optimize the TMA strategy (10%)

In this task, you are asked to optimize the TMA strategy you have implemented in Task 3. Given a cerebro, you need to create a TMA strategy class (by calling the function `task_3()`), add the strategy to the cerebro, and set the sizer. The parameters you optimize are the short period, the medium period, and the long period. The short period can take one of three values: 5, 10, and 20. The medium period can take one of three values: 50, 100, and 150. The long period can take one of three values: 200, 250, and 300. You need to run the cerebro with `maxcpu` set to 1 and see which combination yields the highest profit in the end.

You are asked to return the best parameter combination and the best value you obtained above. Note that the best parameter combination should be the best for your own trading period (from `get_periods.py`). You need to return two objects. The first is your MWS username. The second is a dictionary containing the best parameters. Please use the trading data given by `get_periods.py` using your username. For example:

```
best_result = {}
best_result["short_period"] = 5
best_result["medium_period"] = 50
best_result["long_period"] = 200
best_result["value"] = 95624.37

return "andyrox", best_result
```

Please use the following parameters when running cerebro:

```
data=data,
cash=1e6,
commission=0.001,
slippage_percentage=0.01
stake=10
```

Task 8: Risk_adjusted Ratio (10%)

In this task, you are asked to compute the total compound return and the Sharpe ratio (with a risk-free rate of 0.02) of the Bollinger Band Overbought/Oversold strategy in your trading period. You need to add analyzers and retrieve the values of the analyzers after running the backtesting.

You need to return two objects. The first is your username. The second is the `risk_adjusted_returns` dictionary. Please use the trading data given by `get_periods.py` using your username.

```
returns = {}
returns["total_compound_return"] = xxx
returns["sharpe_ratio"] = xxx

return "YOUR_USERNAME_HERE", returns
```

Please use the following parameters when running cerebro:

```
data=data,
cash=1e6,
commission=0.001,
slippage_percentage=0.01
stake=10
```

Part 2: Pair trading strategy implementation

In this task, you are asked to implement the Pair Trading Strategy using BackTrader from scratch. You need to work with the Python file named `pair_trading_template.py`, where you can initialize a cerebro with your pair trading strategy implementation. As before, you need to change the name to `pair_trading_solution.py` when you submit it to CodeGrade.

We will use two stocks for the pair trading, Pepsi (PEP) and Coca-Cola (KO). In this task, every student has their own trading data periods based on their MWS username (only the part before the @, e.g., for Andrew Roxburgh, this username is `andyrox`, rather than the full email form `andyrox@liverpool.ac.uk`). By having different time periods for different students, there is not one single correct answer for this task. To get your periods, you need to run `get_periods.py` with your username.

```
python get_periods.py YOUR_USERNAME_HERE
```

Running `get_periods.py` will give you a start date and an end date. With these two dates, you can download the adjusted close prices of Pepsi and Coca-Cola, which we will use for the pair trading strategy.

After creating a cerebro, you can feed in the data you have downloaded and processed with `bt.feeds.PandasData`. Note that here you need to add the trading data of two stocks to the cerebro, which is different from what we have done in the previous tasks, where only one stock is used. You need to add each stock separately and assign a name to the stock. Then you can access them properly using `self.datas[0]` and `self.datas[1]` in a cerebro. Let's assume Pepsi will be added first. So `self.datas[0]` is Pepsi.

Then you need to add your implementation of the pair trading strategy to the cerebro. You should implement the pair trading strategy in class `PairTradingStrategy(bt.Strategy)`. The default parameters of the class include `zscore_entry`, `zscore_exit`, and `lookback`. `zscore_entry` and `zscore_exit` will be used for defining the entry and exit conditions based on Z-score, which we will discuss later. `lookback` will be used for computing the hedge ratio.

You are allowed to add whatever variables you would like to define for the class in the `__init__` function and then implement the logic in the `next` function.

The `next` function is split into 4 subproblems as follows.

Extra 1: Compute hedge ratio using linear regression (10%)

In this task, you are asked to employ Ordinary Least Squares (OLS) regression to compute the hedge ratio of the two stocks using the same data time period given by `lookback`. Specifically, you need to apply OLS to the historical data with length `lookback` from the current time (I have already written the data-handling part for you by calling Extra 1 in `next`, so you only need to fill in Extra 1). So the hedge ratio can vary as time proceeds. We will use the following packages and functions:

```
from statsmodels.regression.linear_model import OLS
from statsmodels.tools.tools import add_constant
```

Learn how to use OLS by visiting

https://www.statsmodels.org/dev/generated/statsmodels.regression.linear_model.OLS.html

The output of OLS will be a constant and the hedge ratio. You need to return the constant and the hedge ratio in the form of

```
def extral(prices1, prices2):
    # Fill in your code

    return LS_const, hedge_ratio
```

Extra 2: Compute the spread (2%)

In this task, you are asked to compute the spread using the hedge ratio and constant from Extra 1. The spread should be the price of stock 1 minus the `hedge_ratio` times the price of stock 2 plus the constant. Note that in class, for simplicity, we don't have the constant. But we assume we need it for OLS here. We will use Pepsi as stock 1 and Coca-Cola as stock 2. You need to return the spread for this function.

Extra 3: Compute Z-score (3%)

In this task, you are asked to compute the Z-score. We will use the Z-score as part of the trading logic. Now let's introduce the Z-score.

The Z-score of the spreads is calculated as:

$$z = \frac{\text{Current Spread} - \mu_{\text{Spreads}}}{\sigma_{\text{Spreads}}}$$

Where:

- μ_{Spreads} : mean of the spreads over the lookback period.
- σ_{Spreads} : standard deviation of the spreads over the lookback period.

Given a list of spreads, you need to return the Z-score of the current spread.

Extra 4: Complete the rest and return the final value (15%)

In this task, you need to fill in the rest of the `next` function by implementing the logic of pair trading.

Entry Conditions

The entry conditions define when the strategy initiates a pair trade. The logic is based on the Z-score.

Key Idea: If the Z-score is far from 0, it suggests that the spread between the two stocks has deviated significantly from its historical mean, creating a trading opportunity.

Conditions:

1. Z-Score > Entry Threshold:

- The spread is significantly above its mean.
- Action:
 - Short Stock 1: Stock 1 is relatively overpriced.
 - Long Stock 2: Stock 2 is relatively underpriced.
- Expectation: The spread will revert to the mean, reducing the z-score to zero.

2. Z-Score < -Entry Threshold:

- The spread is significantly below its mean.
- Action:
 - Long Stock 1: Stock 1 is relatively underpriced.
 - Short Stock 2: Stock 2 is relatively overpriced.
- Expectation: The spread will revert to the mean, increasing the z-score to zero.

Exit Conditions

The exit conditions define when to close the pair trade. The logic is based on the z-score returning toward zero.

Key Idea: When the z-score moves close to 0, the spread has reverted to its mean, and the trade has played out.

Condition:

- **Absolute Z-Score < Exit Threshold:**

- The spread is close to its mean, indicating the trade has reached its target.
- Action:
 - * Close both the long and short positions.

Our default parameters are `zscore_entry=2.0`, `zscore_exit=0.1`, and `lookback=30`. You need to set the size of stock 1 to be 100 for each trade. Then the size of stock 2 is the hedge ratio multiplied by 100. Setting the size for a particular trade can be done by setting parameters for `self.sell(data=xxx, size=xxx)` and `self.buy(data=xxx, size=xxx)`. You need to replace `xxx` with the correct values. You can research how to set sizes within `self.buy` and `self.sell` by reading the BackTrader documentation.

Once you implement the pair trading strategy, you need to add the strategy to the cerebro. Besides, you need to set the initial cash to 1000000.0 and the commission to 0.001. Then run your cerebro and see how your pair trading strategy performs. You can do all of this in the main function. You need to return your final value in Extra 4.

Note that you don't need to change the existing code. You need to read the existing code and fill in the gaps.

THE END